

Predicting Mouse Click Position Using Long Short-Term Memory Model Trained by Joint Loss Function

Datong Wei*
Peking University
Beijing, China

Xiaolong Zhang[†]
Pennsylvania State University
Pennsylvania, USA

Chaofan Yang*
Peking University
Beijing, China

Xiaoru Yuan*
Peking University
Beijing, China

ABSTRACT

Knowing where users might click in advance can potentially improve the efficiency of user interaction in desktop user interfaces. In this paper, we propose a machine learning approach to predict mouse click location. Our model, which is LSTM (long short-term memory)-based and trained by joint supervision, can predict the rectangular region of mouse click with feeding mouse trajectories on the fly. Experiment results show that our model can achieve a result of a predicted rectangle area of 58×79 pixels with 92% accuracy, and reduce prediction error when compared with other state-of-the-art prediction methods using a multi-user dataset.

CCS CONCEPTS

• Human-centered computing → Pointing.

KEYWORDS

User Intention, Mouse Interaction, Mouse Prediction, Machine Learning

ACM Reference Format:

Datong Wei, Chaofan Yang, Xiaolong Zhang, and Xiaoru Yuan. 2021. Predicting Mouse Click Position Using Long Short-Term Memory Model Trained by Joint Loss Function. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3411763.3451651>

1 INTRODUCTION

With the advent of graphical user interfaces (GUI), mouse, as an efficient pointing device, has become essential to various computer

systems. Despite the emergence of many other input devices and modalities in recent years, mouse is still a must-have input device for users to interact with GUI-based applications.

Interaction designs and research on mouse control behaviors largely focus on which mouse button is clicked, how the system should respond to a particular click, and what GUI object a click target. For example, Fitts' Law[4], one of the foundation theories in Human-Computer Interaction (HCI), modeled the relationship among the time for a cursor to point to a target, target size, and target distance to the initial position of the cursor, and led to various designs to assist mouse behaviors, such as target magnification[16] and gravity wells[10]. Most of these methods require prior knowledge of users' intentions when they control the mouse.

However, as user interfaces become more and more complex and have more and more actionable interactive objects, simply manipulating target size or distance to help mouse pointing may become impractical. For example, in various visualization designs, the location and size of an object are mapped to important attributes of a data point that the object represents, and any change of its location or size will indicate the change of the value of a particular attribute and misinform users.

A similar challenge exists when users interact with networked systems or applications. Currently, more and more applications become services residing on a server on the internet, rather than on the local machine. As user interactive activities and system responses are separated by the internet, network latency can lead to long waiting time for both users and the system, and consequently degrade user experience.

One way to address this issue from the user side is to predict a user's mouse movement. If the system knows where the cursor movement may end, the system can infer what actionable interactive objects are interesting to the user, and then take actions in advance. Consequently, system delay can be reduced and user experience can be improved.

Some methods [12, 20] have been developed to predict the position of the endpoint. However, they provided no data on the reliability of the prediction results, so using their results may face some challenges. For example, to prefetch data from the server to reduce latency based on predicted actions, a web-based service needs to know to what extent the predicted actions are accurate. Otherwise, incorrect data prefetching may introduce more delay. With information of predicated actions as well as the probabilities of predictions, the service can make better decisions.

*Datong Wei, Chaofan Yang and Xiaoru Yuan are with Key Laboratory of Machine Perception (Ministry of Education) and National Engineering Laboratory for Big Data Analysis and Application, Peking University. Emails: {datong.wei, chaofanyang, xiaoru.yuan}@pku.edu.cn. Xiaoru Yuan is the corresponding author.

[†]College of Information Sciences and Technology, Pennsylvania State University. Email: lzhang@ist.psu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8095-9/21/05...\$15.00

<https://doi.org/10.1145/3411763.3451651>

The advances of deep learning techniques[14, 25] offer opportunities to improve the accuracy of mouse movement prediction. The mouse movement prediction problem can be translated into a time series prediction problem, which can be solved by various deep learning methods, such as Long Short-Term Memory (LSTM) [9].

To obtain the reliability of prediction, it is necessary to make the model output an interval rather than a point. However, in usual machine learning methods, the model output is consistent with the training data, and it is hard to collect mouse click data with uncertain intervals. In this regard, we propose a custom loss function that can still be used to update the model parameters according to the true mouse click positions even if the model output is an interval.

In this paper, we propose a method to predict possible mouse clicking areas in real-time based on LSTM network trained by joint supervision. It uses ongoing mouse trajectories for prediction. The model output is an area where the mouse is likely to click rather than a specific endpoint. The size of the prediction area is associated with a confidence level of the prediction. Experiment results show that the mouse click position lies within our prediction area in 89% of the cases.

The paper is structured as follows. We first review relevant research in Section 2, and then describe our approach in Section 3. Section 4 presents an experiment to collect user mouse behavior data for model building and model comparison. After the discussion of our research and the implications of our findings in Section 5, the paper concludes with future research directions in Section 6.

2 RELATED WORK

Various algorithms have been proposed to predict mouse movement. Generally speaking, these algorithms follow two directions. One direction is target-agnostic mouse click location prediction, which only takes mouse movement data as input and produces a result of the endpoint position on screen. We call this approach position prediction. The other direction is target prediction, which requires the information of clickable targets[2, 13, 17, 19, 26]. The result of this approach is specific targets in an application, so we call it target prediction. Unlike the position prediction methods mentioned above, the inputs of target prediction algorithms must contain information about targets in an application. This requirement makes such algorithms more difficult to deploy. In this section, we focus on position prediction algorithms.

For target-agnostic mouse click position prediction, the input is only the mouse movement information. Algorithms on position prediction can be classified into two categories: algorithms based on regression models, and algorithms based on template matching. The method proposed by Assano et al.[1] used a simple linear regression model to predict where a click may be. The linear regression model was built on the work by Takagi et al.[23], which described the relationship between the peak velocity and the endpoint of a movement, which is assumed as the position to click. A more complicated prediction model by Lank et al. [12] was based on the laws of motion - Kinematic Endpoint Prediction (KEP). This model consists of two steps: 1) creating a curve of speed versus distance by using mouse movement data, and 2) fitting the curve to a quadratic

equation obtained a prior with the least squares regression. Their experimental results showed that the KEP algorithm can make correct prediction with a probability of 42% when a movement is 80% completion. This algorithm was further simplified by Ruiz and Lank [21], which also proposed a technique to measure the stability of position prediction.

Algorithms based on template matching rely on a template library, rather than a model, to predict endpoint. The prediction method by Pasqual et al.[20], Kinematic Template Matching (KTM), is an example of such algorithms. KTM uses the relationships between speed and time during mouse movement as templates, and finds the template that best matches a user's mouse movement. With the template and mouse movement data, the final position can be calculated. The KTM method has been applied in ray pointing prediction in VR[8]. Although KTM produces smaller errors than KEP, its generability is limited, because KTM templates are user dependent and demand large amount of user mouse behavior data for template building. Both KTM and KEP don't provide prediction confidence, which limits their practical application.

3 METHOD

In this section, we present our method to predict possible mouse clicking region. In this work, we view mouse movement trajectories as time-series data, so possible clicking region prediction becomes a forecasting task for time series data. Here, we first introduce the LSTM approach, which is effective in modeling time-series patterns and making predictions, and then describe our LSTM implementation for mouse clicking region prediction.

3.1 Definitions

Before describing the overall pipeline of our LSTM implementation, we first define some involved concepts.

LSTM [9] is a type of RNN[3], which has been widely applied nowadays due to its strong capabilities for feature modeling and generalization[15]. LSTM improves the capacity of RNN in remembering past information. It is capable of handling motion sequences with unfixed lengths. Mouse movement can be encoded as a sequence input to an LSTM model, while possible mouse clicking region be the output of the model. LSTM captures features in the mouse movement sequence and learns the implicit mapping of the mouse movement sequence to the possible region in which it ends.

Mouse movement is defined as a set of data points, each of which has a time stamp and a pair of 2-dimensional coordinates, denoted as: $[t_0, (x_0, y_0), t_1, (x_1, y_1), t_2, (x_2, y_2), \dots, t_n, (x_n, y_n)]$. If mouse positions are sampled with a constant frequency, the time stamp component can be omitted: $[(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$

Mouse endpoint is a 2-dimensional point where mouse click happens. Following the above definition of mouse movement, the endpoint is (x_n, y_n) .

Possible mouse clicking region is a 2-dimensional rectangular region in which click might happens. The region is defined by 4 boundaries: upper and lower bound in both x and y axis denoted by $[(x_{lower}, y_{lower}), (x_{upper}, y_{upper})]$

The input for our proposed algorithm is $[(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)]$, $k < n$ and output is $[(x_{lower}, y_{lower}), (x_{upper}, y_{upper})]$.

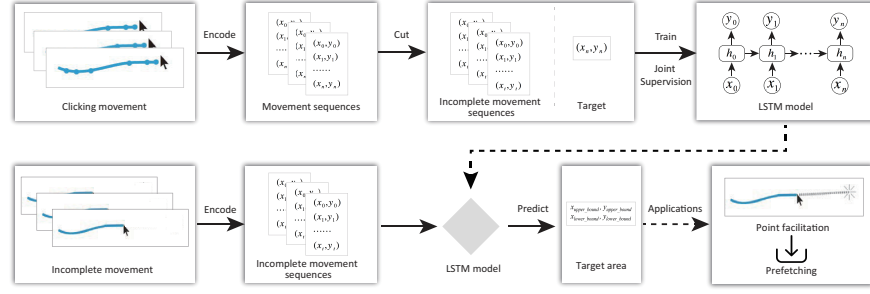


Figure 1: Pipeline of mouse clicking region prediction with LSTM model.

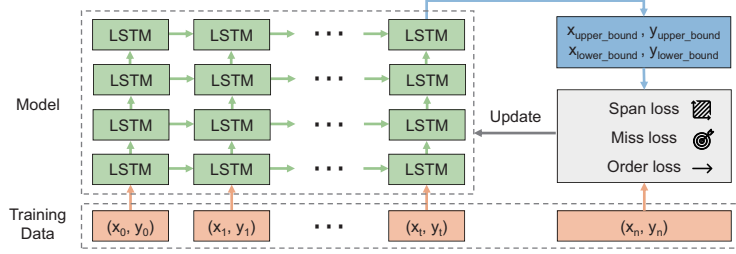


Figure 2: Architecture of the LSTM model used for region prediction. Green horizontal arrows represent the model unfolded in time.

3.2 Joint Loss Function

The data structures of training data and model output are not identical, so usual loss functions such as Mean Absolute Error (MAE) can't be used. We propose a joint loss function consisting of the following three loss functions for model training.

Span loss is the span of prediction region, and it penalizes the model when the span of the prediction region is large. Span loss of x dimension is calculated as $L_{span_x} = |x_{upper} - x_{lower}|$. Span loss is composed of losses of both x and y dimensions: $L_{span} = L_{span_x} + L_{span_y}$, and the same is true for miss loss and order loss.

Miss loss penalizes the model when the true end point lay out of the output region:

$$L_{misscoef_x} = \left(\frac{|(x_{upper} - x_n) \cdot (x_{lower} - x_n)|}{(x_{upper} - x_n) \cdot (x_{lower} - x_n)} + 1 \right),$$

$$L_{miss_x} = L_{misscoef_x} \cdot |(x_{upper} - x_n) \cdot (x_{lower} - x_n)|.$$

x_n stands for the ground truth mouse endpoint in x dimension. The result of the fraction is 0 when the denominator is 0, the same is true for order loss.

Order loss penalizes the model when the lower bound of the output is greater than the upper bound:

$$L_{order_x} = \left(\frac{|x_{lower} - x_{upper}|}{x_{lower} - x_{upper}} + 1 \right) \cdot |x_{upper} - x_{lower}|.$$

The formula of the joint loss function is $L = \alpha \cdot L_{span} + \beta \cdot L_{miss} + \theta \cdot L_{order}$, $\alpha < \beta < \gamma$.

3.3 LSTM Model for Region Prediction

Figure 2 shows the architecture of our proposed model. It is composed of 4 layers, and the number of LSTM units in each layer are

20, 40, 20, and 1, respectively. The hidden layers are fully connected. The model takes mouse position sequences as input, and output the possible clicking regions.

The LSTM model is implemented with Python, and the deep learning framework is Keras, which is based on Tensorflow. We tested different coefficients for joint loss function described in Section 3.2 and found that $\alpha = 0.01$, $\beta = 1$ and $\theta = 100$ is a suitable setting. Training data of the model includes incomplete mouse trajectories and their corresponding endpoints. More details on training data are provided in Section 4. The model is trained with the Back Propagation Through Time (BPTT) method[22, 24] and the Adam optimizer[11].

During the training process, we divide the dataset into a training set and a validation set. While the training set is used to train the model, the validation set helps to monitor the change of the loss function value and prevent overfitting.

4 EXPERIMENT

We conducted a controlled experiment to collect data for model building and evaluate the predictive accuracy of our model.

4.1 Participants and Apparatus

Eight participants (age between 22 and 25) were recruited for this study. They came from both college and industry, and had diverse backgrounds. All of them were right-handed and indicated that they were familiar with computers.

Tasks were performed in a desktop PC, with a 21.5 inch screen set to a resolution of 1920×1080 pixels. A wired optical mouse was used as the input device under the system default mouse settings.

We developed a web-based application with JavaScript to capture mouse movement data. The application was run on Google Chrome.

4.2 Tasks

The tasks were similar to those used to evaluate the algorithm performance on mouse endpoint prediction [20]. Participants were asked to click a series of 500 circular objects, the radius of which was 16 pixels. All these targets were randomly distributed within a circular ring with an inner radius of 50 pixels and an outer radius of 400 pixels (Figure 3). The appearing sequence of the 500 targets was a variant of ISO 9241-9[5, 6].

The targets were shown one by one. A new target appeared only after the click action was made to the previous target, again regardless whether the hit was accurate or not.

4.3 Procedure

In each trial, we first briefed what participants needed to accomplish and answered their questions. Considering the fact that clicking is a common task people perform regularly, we did not offer task practice and asked participants to start a trial after the brief.

For each participant, clicking tasks were divided into several stages and each stage contained 25 trials. To start a stage, participants must click a 'Start' button located at the upper left corner of the screen. After the click, the button disappeared and the first target was shown. The first target was right above the center of the screen with a random distance to the center between 50 to 400 pixels. The mouse movement from the 'Start' button to the first target was not recorded.

The total time for a participant to complete all tasks was about 25 minutes on average.

4.4 Mouse Movement Data Processing

Data collected from the experiment has to be processed first before used for modeling training. In this section, we provide an overview of the data and explain the data processing procedure.

4.4.1 Data Overview. Participants completed a total of 4284 mouse clicks and the rate of a successful hit was 92.0%. Among all these clicks, 32.3% of mouse movements overshoot the target. This means that overshooting is common in mouse movements, so we included all these movements as valid data rather than removing them. Those mouse movements that did not hit the target were dropped. Eventually, we collected a total of 3942 valid movements.

We examined the distributions of target distance and task completion time. Because of the back-and-forth paths of mouse movements for target clicking and the random target positions in our experiment, the distribution of target distance is the sum of two independent uniform distributions. The distribution of task completion time is close to a Gaussian. The average task completion time is 700 ms, and the range is between 400 and 1000 ms. Only 8.7% of the data had a task completion time longer than 1000 ms and only 6.7% of the data shorter than 400ms.

4.4.2 Data Processing. The sampling time of mouse position sequences was not evenly distributed due to the limitation of the web application, so we re-sampled the sequences with a sampling interval of 20ms and linearly interpolated the two data points adjacent

to the sampling time points to obtain new sampling points. We compared position sequences after re-sampling and did not find significant difference between raw and re-sampled data.

Then, we randomly assigned the complete mouse trajectories to two separate sets in a 4:1 ratio: 80% of mouse trajectories for the training set and the rest 20% for the test set. The training set was used to train our LSTM model and the test set to evaluate the performance of our algorithm. Thus, 3109 mouse movements were assigned into training set while 833 into the test set.

We processed the training data for our LSTM model with the following steps:

- (1) **Normalization:** Having all mouse movements start at 0 and end at (x,y), with both x and y being positive and the range of mouse position coordinates being between 0 and 1 under a global scaling factor.
- (2) **Cutting:** Obtaining an incomplete trajectory from each complete mouse trajectory. The minimum time span of an complete trajectory is 200 ms, and the maximum is 1000 ms. The increment is 20 ms. For mouse movements shorter than 200 ms, no segment would be created. For mouse movements longer than 1000 ms, 50 segments were taken. For mouse movements between 200ms and 1000ms, the number of segments increases linearly with movement time. Under this cutting approach, slow mouse movements have greater weights in the training data.

After the above operation, we intercepted 79998 incomplete motion trajectories from 3109 complete motion trajectories as the model input of the training data, and their corresponding endpoints are the model output.

4.5 Result

To compare with previous algorithms, we calculated the error based on the distance between the actual click position and the center of the predicted rectangular region. We compared our LSTM model with KEP[12] and KTM[20]. We implemented the KEP and KTM algorithms by using Python. The KEP algorithm needs no training data, while the KTM algorithm requires full mouse movement to build a template library. We used full mouse movements of the LSTM training data to build the kinematic template library. It is worth noting that we mainly wanted to test the generalizability of our model for different users, so the training data was composed of mouse movement trajectories from different users. This approach differs from the initial implementation of the KTM algorithm, which used data from one user, and may lead to lower than expected performance by KTM.

All these three algorithms were tested using the same incomplete mouse movement dataset. The sampling of the test data was consistent with that of the training data, again based on equal intervals with a minimum time span of 200 ms.

Our LSTM model achieves the best performance of the three. When mouse movement is 70% completion in terms of time (Figure 5), the average errors by KEP, KTM, and LSTM are 40, 60, and 26 pixels, respectively. From the perspective of distance percentage (Figure 4), at the point of 70% completion the errors by KEP, KTM, and LSTM are 129, 115, and 68 pixels, respectively.

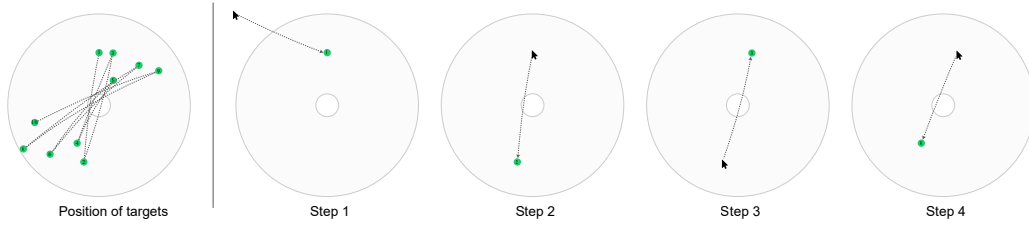


Figure 3: Illustration of clicking task. Circular targets are shown in the order indicated by the lines and numbers. The number of targets are reduced for better interpretation.

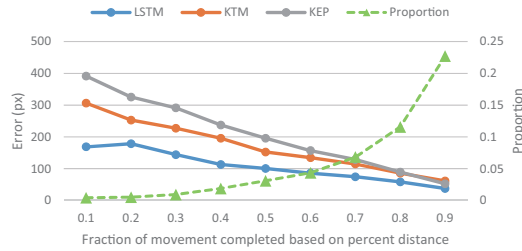


Figure 4: Error of mouse endpoint prediction aggregated by distance percentage. The green dashed line indicates the proportion of the corresponding movement fraction in the test data.

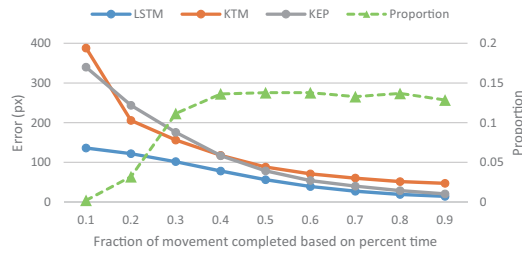


Figure 5: Error of mouse endpoint prediction aggregated by time percentage. The green dashed line indicates the proportion of the corresponding movement fraction in the test data.

The error of LSTM is significantly smaller than that of KEP and KTM according to a Wilcoxon signed-rank test ($p < 0.01$) from 10% to 80% completion in terms of time. At 90% completion in terms of time, the error of LSTM is significantly smaller than the error of KTM ($p < 0.01$), but not significantly smaller than that of KEP ($p = 0.15$). When it comes to distance completion, LSTM achieves better performance from 10% to 90% completion compared with the other two algorithms ($p < 0.01$).

We also evaluated the hit rate of the predicted region as well as the length and width of the region, as shown in Figure 6 and Figure 7. The hit rate stays above 84%. And the hit rate is 96% when the percentage of time is 90%, meanwhile, the span in the x-direction is 48 pixels and the span in the y-direction is 72 pixels.

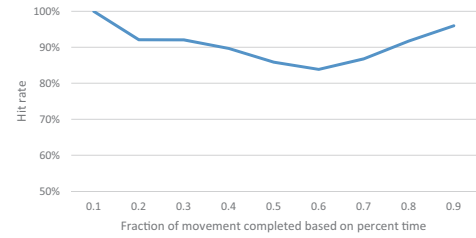


Figure 6: Hit rate of predicted region aggregated by time percentage.

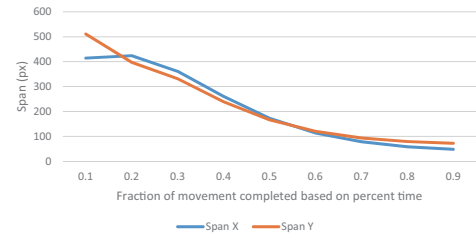


Figure 7: Span of predicted region aggregated by time percentage.

5 DISCUSSION

Experimental results show that the prediction error of our LSTM model is significantly lower than that of previous works. The use of the LSTM model to assist user click interaction has considerable application promise. For example, when a click operation requires longer than expected responding time in situations like using the mouse to interact with a wall-size display or using a networked service with long network latency, accurate prediction of mouse movement endpoints allows the system to take certain actions in advance, such as pre-loading a dataset or pre-sending a network request, to improve user experience.

Of the past studies on enhanced mouse interaction, few studies has used deep learning models. Deep learning algorithms perform better on various tasks compared to traditional algorithms, although they require more computational resources and may be difficult to be applied in certain real-time, demanding scenarios. However, with the popularity of deep learning, various acceleration solutions [18] have been proposed and some complex deep learning models can even run on mobile devices[7]. Real-time interaction

optimization based on deep learning becomes possible. In our tests, running on a regular desktop with an Intel i7-6700K CPU and 16GB RAM without GPU acceleration on, our model can predict from 20000 mouse movement sequences within about 20 seconds, or 1 ms for each prediction. We believe that the computation power of regular computers like laptops can meet the requirement of prediction in real time with our LSTM model.

Training LSTM models requires a lot of data, similar to what traditional template matching algorithms do. The difference is that the parameters and details of a template matching algorithm often require human involvement, whereas deep learning algorithms automatically update parameters based on back propagation and gradient descent. In addition, template matching algorithms can only find matches from a template library, which has only a limited collection of templates, while the LSTM model, as a continuous function, can supply unlimited results.

6 CONCLUSION

In this paper, we proposed a deep learning approach for mouse movement prediction. Our LSTM model can predict possible regions where user may click with feeding mouse trajectories on the fly. Experiment results show that compared with state-of-the-art prediction methods, our approach can reduce prediction error rate by over 30%.

Our model can be further improved in the future. First, although the current model can give region prediction, the distribution of possibility is unknown. We will improve our model to provide such information so that people who use the model can better understand prediction results. Second, we will test our model in real applications. We plan to integrate the model into web-based applications. Doing so, we can collect user behavior data in the wild, which in turn can offer insight into the improvement of our model.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers for their valuable comments. This work is supported by NSFC No. 61872013.

REFERENCES

- [1] Takeshi Asano, Ehud Sharlin, Yoshifumi Kitamura, Kazuki Takashima, and Fumio Kishino. 2005. Predictive Interaction Using the Delphian Desktop. In *Proceedings of the 18th Annual ACM Symposium on User Interface Software and Technology* (Seattle, WA, USA) (UIST '05). ACM, New York, NY, USA, 133–141.
- [2] Gökçen Aslan Aydemir, Patrick M Langdon, and Simon Godsill. 2013. User target intention recognition from cursor position using kalman filter. In *Proceedings of the International Conference on Universal Access in Human-Computer Interaction*. Springer, Springer, Las Vegas, NV, USA, 419–426.
- [3] Jeffrey L Elman. 1990. Finding structure in time. *Cognitive science* 14, 2 (1990), 179–211.
- [4] Paul M Fitts. 1954. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of experimental psychology* 47, 6 (1954), 381.
- [5] International Organisation for Standardisation. 2000. *ISO: Ergonomic requirements for office work with visual display terminals (VDTs) - part 9: Requirements for non-keyboard input devices (ISO 9241-9)*. Technical Report. International Organisation for Standardisation.
- [6] International Organisation for Standardisation. 2012. *ISO: Evaluation methods for the design of physical input devices - ISO/TC 9241-411: 2012(e)*. Technical Report. International Organisation for Standardisation.
- [7] Vinayak Gokhale, Jonghoon Jin, Aysegül Dundar, Berin Martini, and Eugenio Culurciello. 2014. A 240 g-ops/s mobile coprocessor for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. IEEE, Columbus, OH, USA, 682–687.
- [8] Rorik Henrikson, Tovi Grossman, Sean Trowbridge, Daniel Wigdor, and Hrvoje Benko. 2020. Head-Coupled Kinematic Template Matching: A Prediction Model for Ray Pointing in VR. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3313831.3376489>
- [9] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [10] Faustina Hwang, Simeon Keates, Patrick Langdon, and P. John Clarkson. 2003. Multiple Haptic Targets for Motion-Impaired Computer Users. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Ft. Lauderdale, Florida, USA) (CHI '03). Association for Computing Machinery, New York, NY, USA, 41–48. <https://doi.org/10.1145/642611.642620>
- [11] Diederik P. Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG]
- [12] Edward Lank, Yi-Chun Nikko Cheng, and Jaime Ruiz. 2007. Endpoint Prediction Using Motion Kinematics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) (CHI '07). ACM, New York, NY, USA, 637–646.
- [13] S. Lee, R. Ha, and H. Cha. 2019. Click Sequence Prediction in Android Mobile Applications. *IEEE Transactions on Human-Machine Systems* 49, 3 (2019), 278–289. <https://doi.org/10.1109/THMS.2018.2868806>
- [14] Yang Li, Samy Bengio, and Gilles Bailly. 2018. Predicting Human Performance in Vertical Menu Selection Using Deep Learning. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–7. <https://doi.org/10.1145/3173574.3173603>
- [15] Junhua Mao, Wei Xu, Yi Yang, Jiang Wang, Zhiheng Huang, and Alan Yuille. 2014. Deep captioning with multimodal recurrent neural networks (m-rnn). In *International Conference on Learning Representations*. ICLR, San Diego, CA, USA, 1–17.
- [16] Michael McGuffin and Ravin Balakrishnan. 2002. Acquisition of Expanding Targets. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Minneapolis, Minnesota, USA) (CHI '02). Association for Computing Machinery, New York, NY, USA, 57–64. <https://doi.org/10.1145/503376.503388>
- [17] Atsuo Murata. 1998. Improvement of Pointing Time by Predicting Targets in Pointing With a PC Mouse. *International Journal of Human-Computer Interaction* 10, 1 (1998), 23–32.
- [18] Daniel Neil, Michael Pfeiffer, and Shih-Chii Liu. 2016. Phased lstm: Accelerating recurrent network training for long or event-based sequences. In *Advances in neural information processing systems*. NeurIPS, Barcelona, Spain, 3882–3890.
- [19] Alvvita Ottley, Roman Garnett, and Ran Wan. 2019. Follow The Clicks: Learning and Anticipating Mouse Interactions During Exploratory Data Analysis. *Computer Graphics Forum* 38, 3 (2019), 41–52. <https://doi.org/10.1111/cgf.13670> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13670
- [20] Phillip T. Pasqual and Jacob O. Wobbrock. 2014. Mouse Pointing Endpoint Prediction Using Kinematic Template Matching. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (CHI '14). ACM, New York, NY, USA, 743–752.
- [21] Jaime Ruiz and Edward Lank. 2009. *Effects of target size and distance on kinematic endpoint prediction*. Technical Report. Cheriton School of Computer Science. Waterloo, Ontario: University of Waterloo.
- [22] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. 1986. *Learning Internal Representations by Error Propagation*. MIT Press, Cambridge, MA, USA, 318–362.
- [23] Ryo Takagi, Yoshifumi Kitamura, Satoshi Naito, and Fumio Kishino. 2002. A fundamental study on error-corrective feedback movement in a positioning task. In *Proc. of Asian Pacific Computer Human Interaction*. ACM, Beijing, China, 160–172.
- [24] P. J. Werbos. 1990. Backpropagation through time: what it does and how to do it. *Proc. IEEE* 78, 10 (1990), 1550–1560.
- [25] Arianna Yuan and Yang Li. 2020. Modeling Human Visual Search Performance on Realistic Webpages Using Analytical and Deep Learning Methods. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3313831.3376870>
- [26] Brian Ziebart, Anind Dey, and J. Andrew Bagnell. 2012. Probabilistic Pointing Target Prediction via Inverse Optimal Control. In *Proceedings of the 2012 ACM International Conference on Intelligent User Interfaces* (Lisbon, Portugal) (IUI '12). ACM, New York, NY, USA, 1–10.